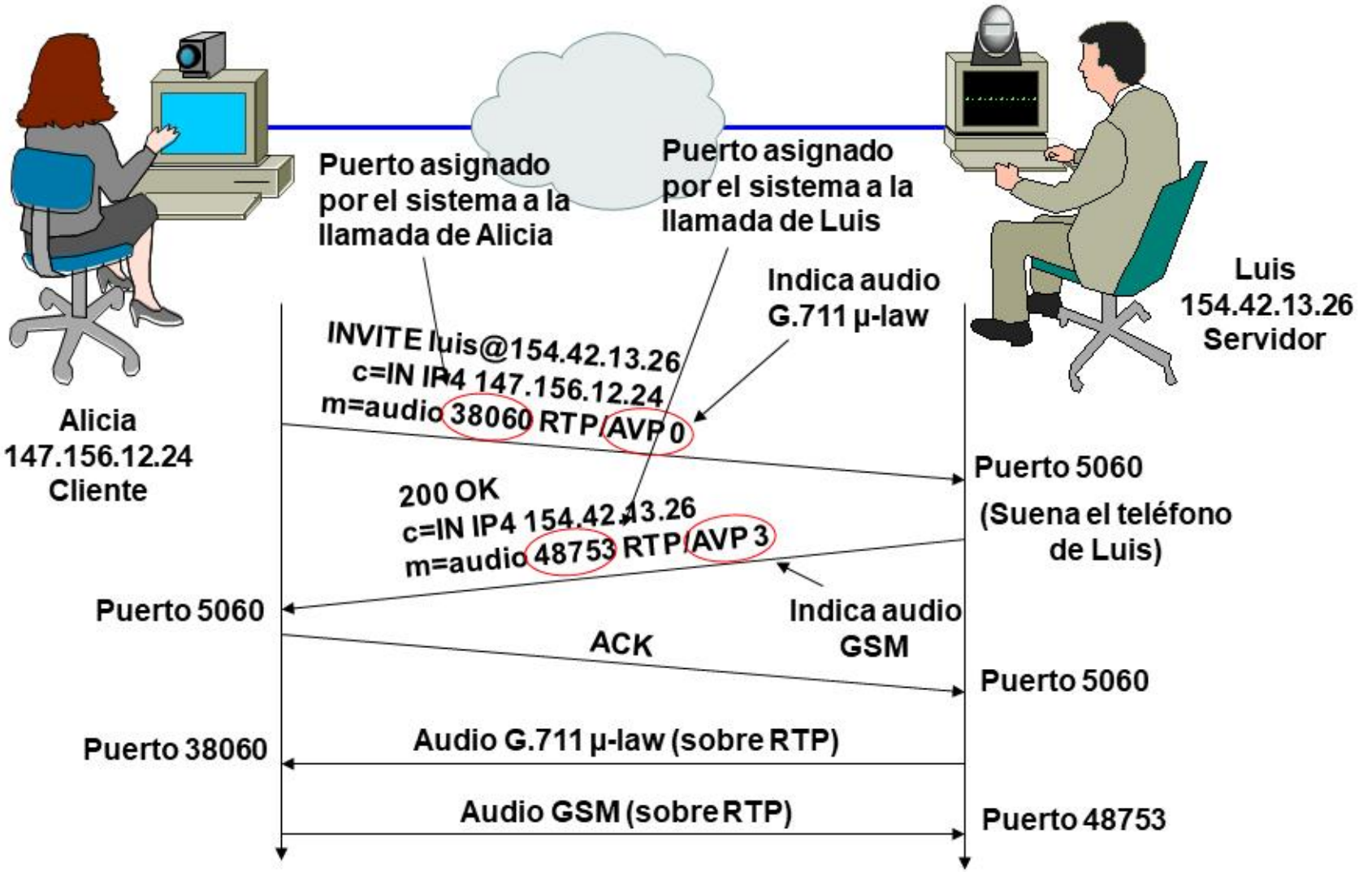
Llamada SIP entre dos usuarios

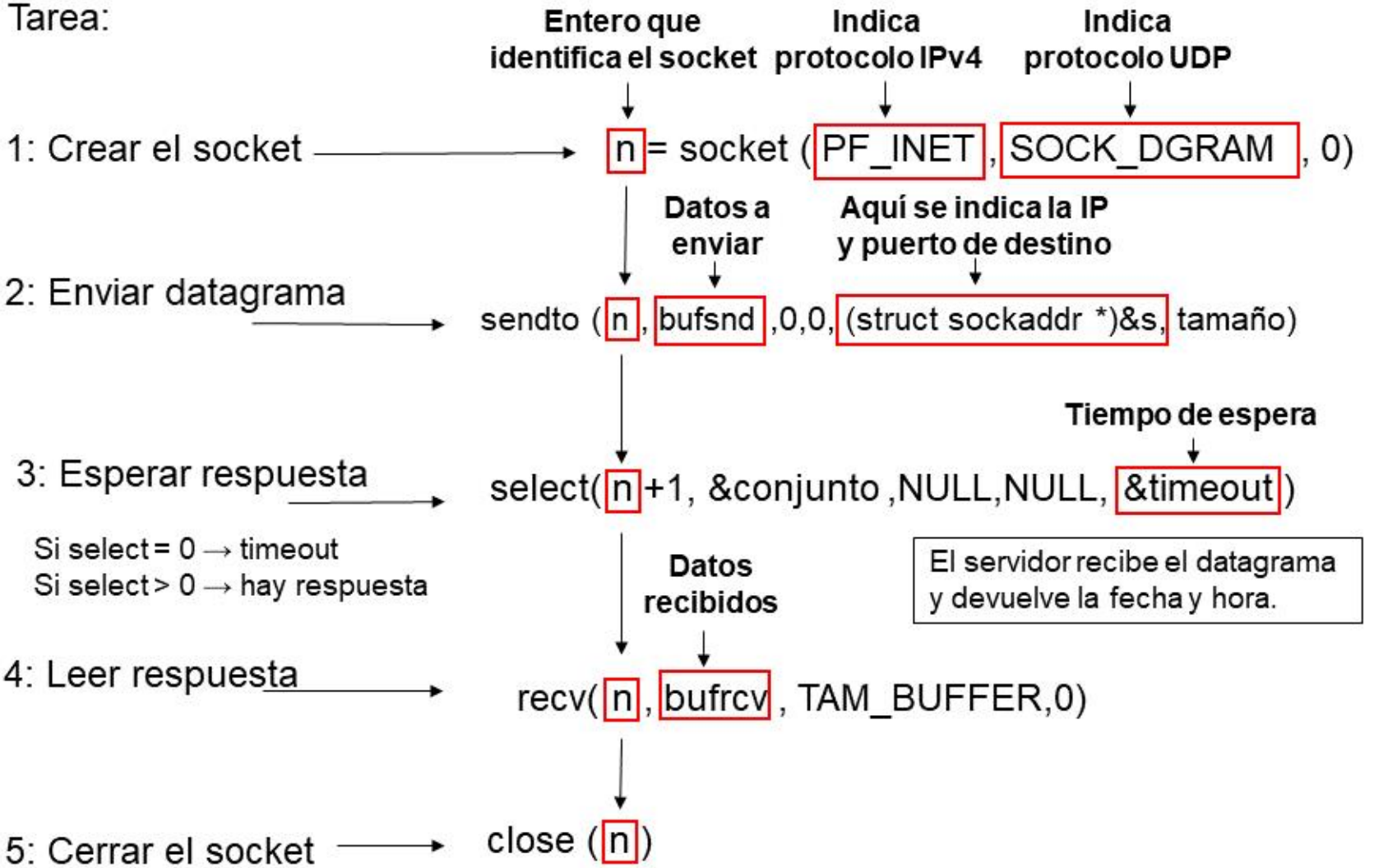


Programación de aplicaciones

- Normalmente se utilizan APIs (Application Programming Interface) estandarizadas denominadas 'sockets'.
- La más extendida son los 'sockets de Berkeley', librería de funciones en C incluída en el UNIX BSD 4.2 en 1983 y posteriormente en el estándar POSIX. Hay versiones para casi todos los S.O., incluido Windows (winsock)
- Estas librerías simplifican las tareas más habituales y permiten al programador escribir código portable sin preocuparse de la arquitectura del host
- La estandarización de APIs se hace por portabilidad, no por interoperabilidad. Lo único que intercambian las aplicaciones son paquetes UDP o TCP

Cliente UDP con Berkeley sockets

Tarea:



Pregunta: tamaño de segmentos

P: El tamaño máximo de un segmento TCP es 65.515 octetos. ¿Podría explicar de donde viene ese valor?

R: La longitud máxima de un datagrama se especifica en un campo de 16 bits, por lo que es 65535 octetos. De estos al menos 20 son la cabecera IP, con lo que quedan 65515 octetos para el segmento TCP.

Pregunta

P: ¿De que manera afecta el tamaño del número de secuencia al caudal máximo de una conexión TCP?

R: Para evitar ambigüedades el número de secuencia no debe darse la vuelta en el tiempo de vida de un segmento. Por tanto con un TTL de 120 el caudal máximo sería $2^{32} = 4.294.967.296$ bytes en 120 segundos:

$$4.294.967.296 \times 8 / 120 = \mathbf{286\ Mb/s}$$

Con un TTL de 255 el caudal sería de solo 135 Mb/s

Cabeceras TCP del inicio de una conexión Telnet

```
1. SYN
TCP:  --- TCP header  ---
TCP:
TCP: Source port = 2345
TCP: Dest port = 23 (Telnet)
TCP: Initial seq. Number = 16421121

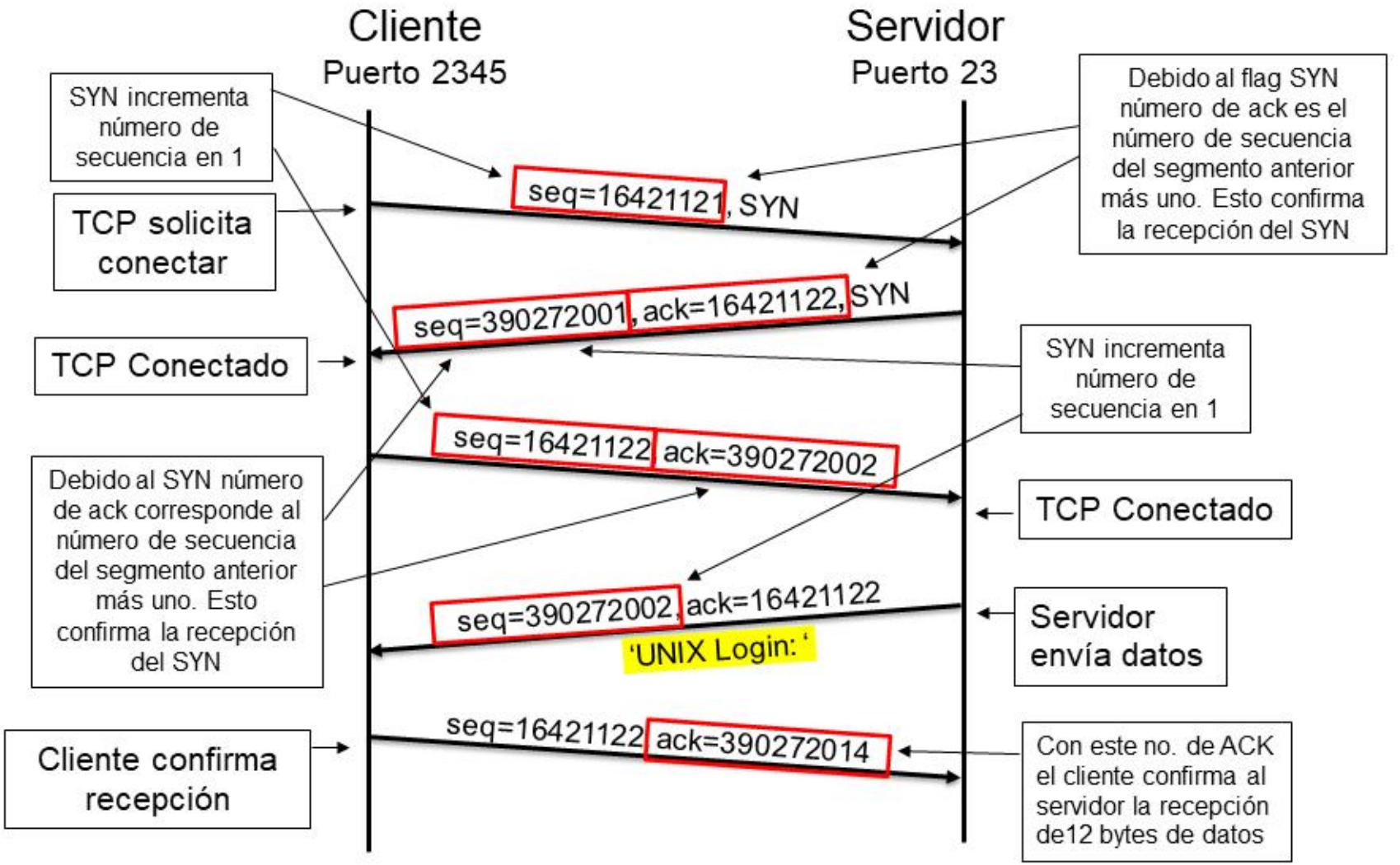
TCP: Data offset = 24 bytes
TCP: Flags = 02 (SYN)
TCP: Window = 2048
TCP: Checksum = F2DA (correct)
TCP:
TCP: Options follow
TCP: Max segment size = 1460
```

```
2. SYN
TCP:  --- TCP header  --
TCP:
TCP: Source port = 23 (Telnet)
TCP: Dest port = 2345
TCP: Initial seq. Number = 390272001
TCP: Acknowledgment Number = 16421122
TCP: Data offset = 24 bytes
TCP: Flags = 12 (ACK,SYN)
TCP: Window = 4096
TCP: Checksum = C13A (correct)
TCP:
TCP: Options follow
TCP: Max segment size = 1024
```

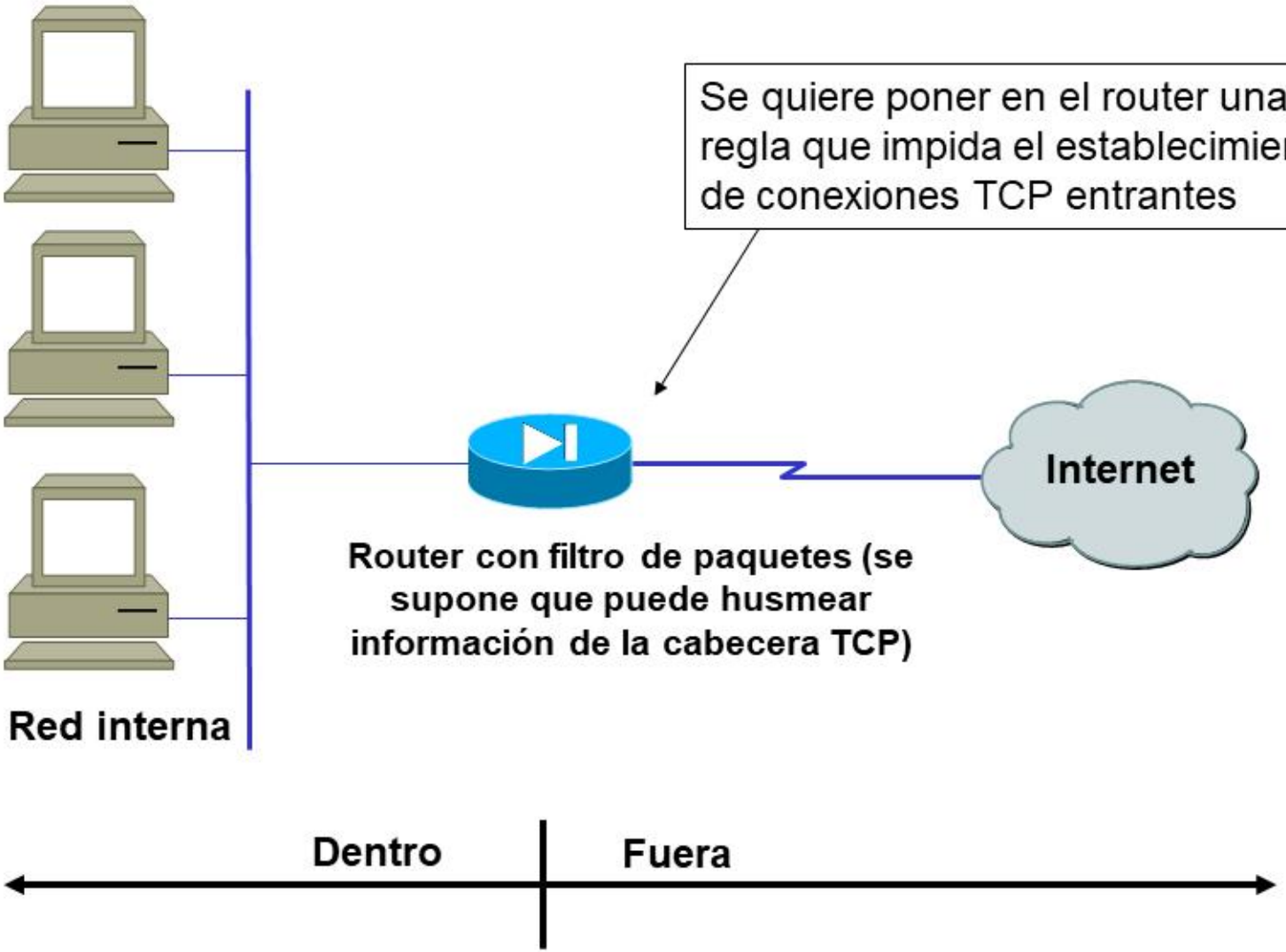
```
3. ACK
TCP:  --- TCP header  ---
TCP:
TCP: Source port = 2345
TCP: Dest port = 23 (Telnet)
TCP: Seq. Number = 16421122
TCP: Acknowledgment Number =
     390272002
TCP: Data offset = 20 bytes
TCP: Flags = 10 (ACK)
TCP: Window = 2048
TCP: Checksum = DF43 (correct)
TCP: No TCP options
```

```
4. DATA
TCP:  --- TCP header  ---
TCP:
TCP: Source port = 23 (Telnet)
TCP: Dest port = 2345
TCP: Seq. Number = 390272002
TCP: Acknowledgment Number = 16421122
TCP: Data offset = 20 bytes
TCP: Flags = 18 (ACK,PSH)
TCP: Window = 4096
TCP: Checksum = 9FEF (correct)
TCP: No TCP options
TCP: [12 byte(s) of data]
```

Intercambio de segmentos del caso anterior



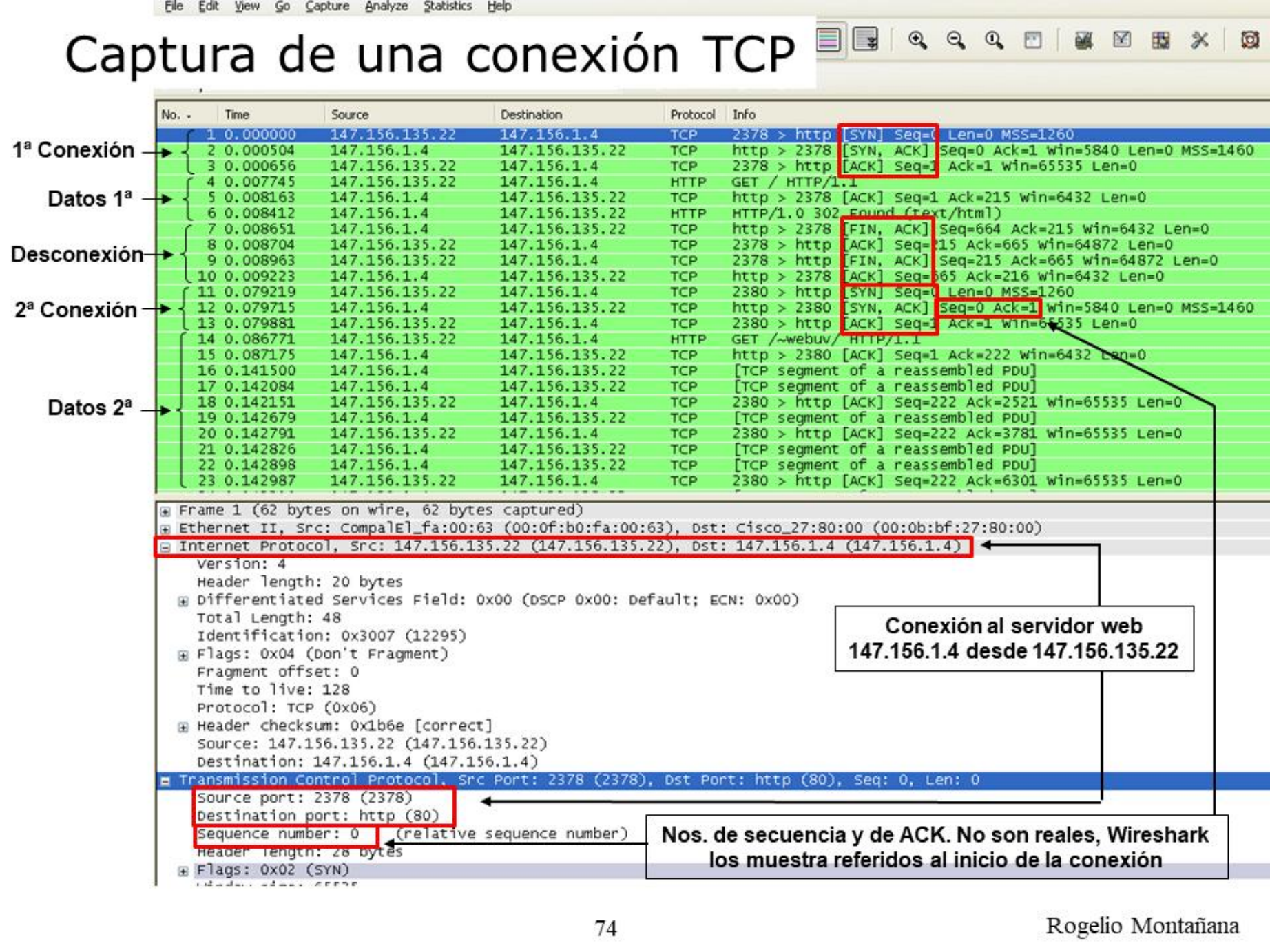
Ejercicio: Cortafuegos básico



Ejercicio: solución

- El router descartará los datagramas que entren por la interfaz serie y que cumplan estas dos condiciones:
  - Tener el valor 6 (TCP) en el campo protocolo de la cabecera IP, y
  - Tener a 1 el flag SYN y a 0 el ACK en la cabecera TCP.
- El segmento que solicita establecer la conexión (el primero de todos) es el único de toda la sesión TCP que tiene la combinación SYN = 1, ACK = 0

Captura de una conexión TCP



Pregunta: fragmentación en TCP

- P: ¿Debe TCP preocuparse de reordenar los fragmentos de un datagrama?
- R: NO. El nivel IP reconstruye el datagrama original antes de entregar el segmento a TCP; para ello usa los campos identificación, fragment offset y MF (More Fragments). TCP no podría reordenar los fragmentos aunque quisiera, puesto que normalmente solo el primero tiene cabecera TCP.

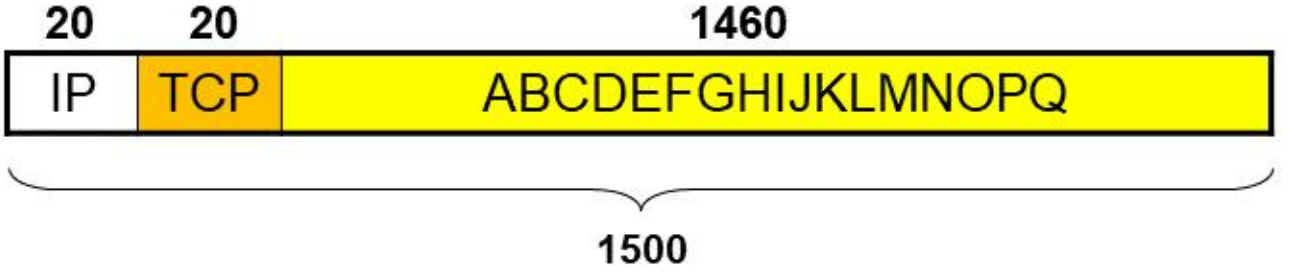
Ejercicio: fragmentación en TCP

- Tenemos un TCP que envía segmentos del tamaño máximo permitido
- El host tiene una interfaz Ethernet (MTU 1500 bytes)
- El otro TCP ha anunciado un MSS de 1460 bytes.
- No se utiliza 'descubrimiento de la MTU del trayecto'
- Sabemos que la ruta pasa por una red con una MTU de 760 bytes
- Indique cuantos datagramas y bytes recibe el nivel de red en el host de destino por cada segmento enviado

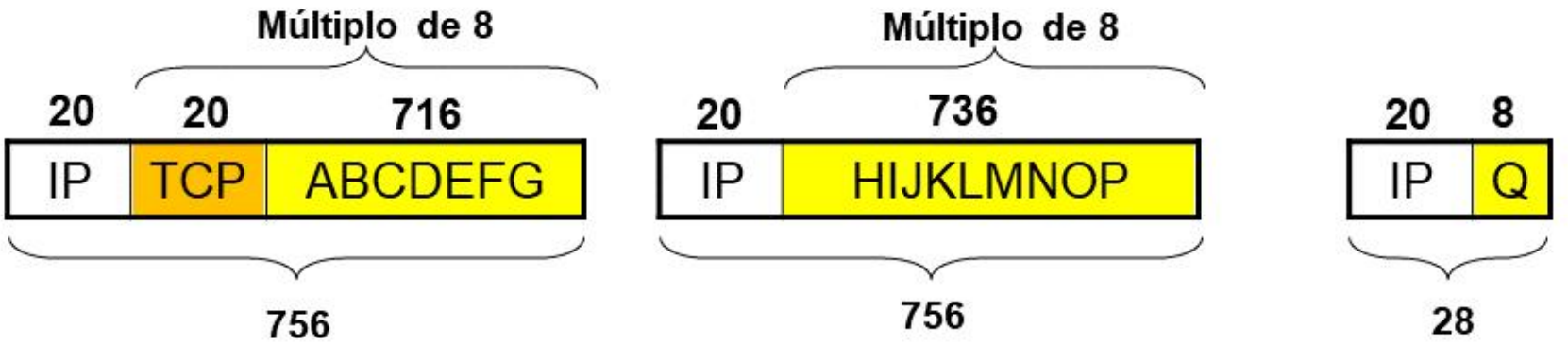
Ejercicio: solución

MSS: 1460 -> Segmento TCP: 1480 -> Datagrama IP: 1500
MTU: 760 bytes

Datagrama emitido por el host:



Fragmentos creados por el router:



Ejemplo de tráfico interactivo: caso telnet

- Telnet (Teletype Network) es un protocolo de nivel de aplicación para la emulación de terminal remoto.
- Para su correcto funcionamiento telnet necesita generalmente utilizar el eco remoto, es decir que los caracteres escritos en el teclado sean representados en la pantalla por el host remoto.
- Esto significa que cada vez que se pulsa una tecla se transmiten como mínimo dos paquetes, uno de ida con el carácter pulsado y otro de vuelta con el carácter que aparecerá en pantalla. Esto es preciso para que funcionen cosas como la ocultación de contraseñas.

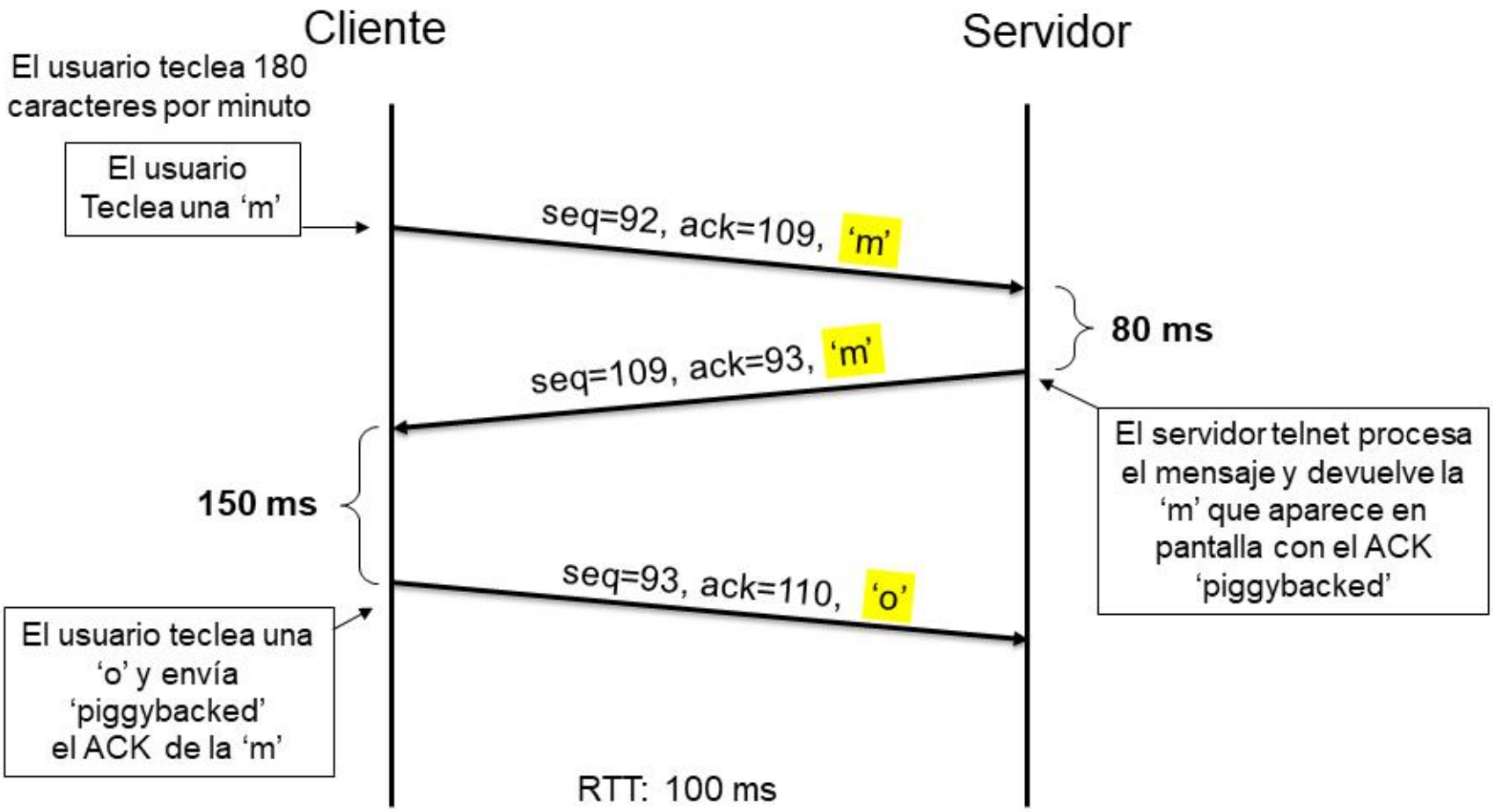
Tráfico interactivo TelNet

- En realidad, por cada tecla pulsada telnet intercambia 2, 3 ó 4 segmentos, dependiendo de la rapidez relativa del usuario (el cliente) y del servidor:

Caso 1: Usuario y servidor rápidos	2 segmentos por pulsación
Caso 2: Usuario lento, servidor rápido	3 segmentos por pulsación
Caso 3: Usuario rápido, servidor lento	3 segmentos por pulsación
Caso 4: Usuario y servidor lentos	4 segmentos por pulsación

Se considera rápido si el tiempo de respuesta es de 200 ms o menor (equivalente a 300 pulsaciones por minuto)

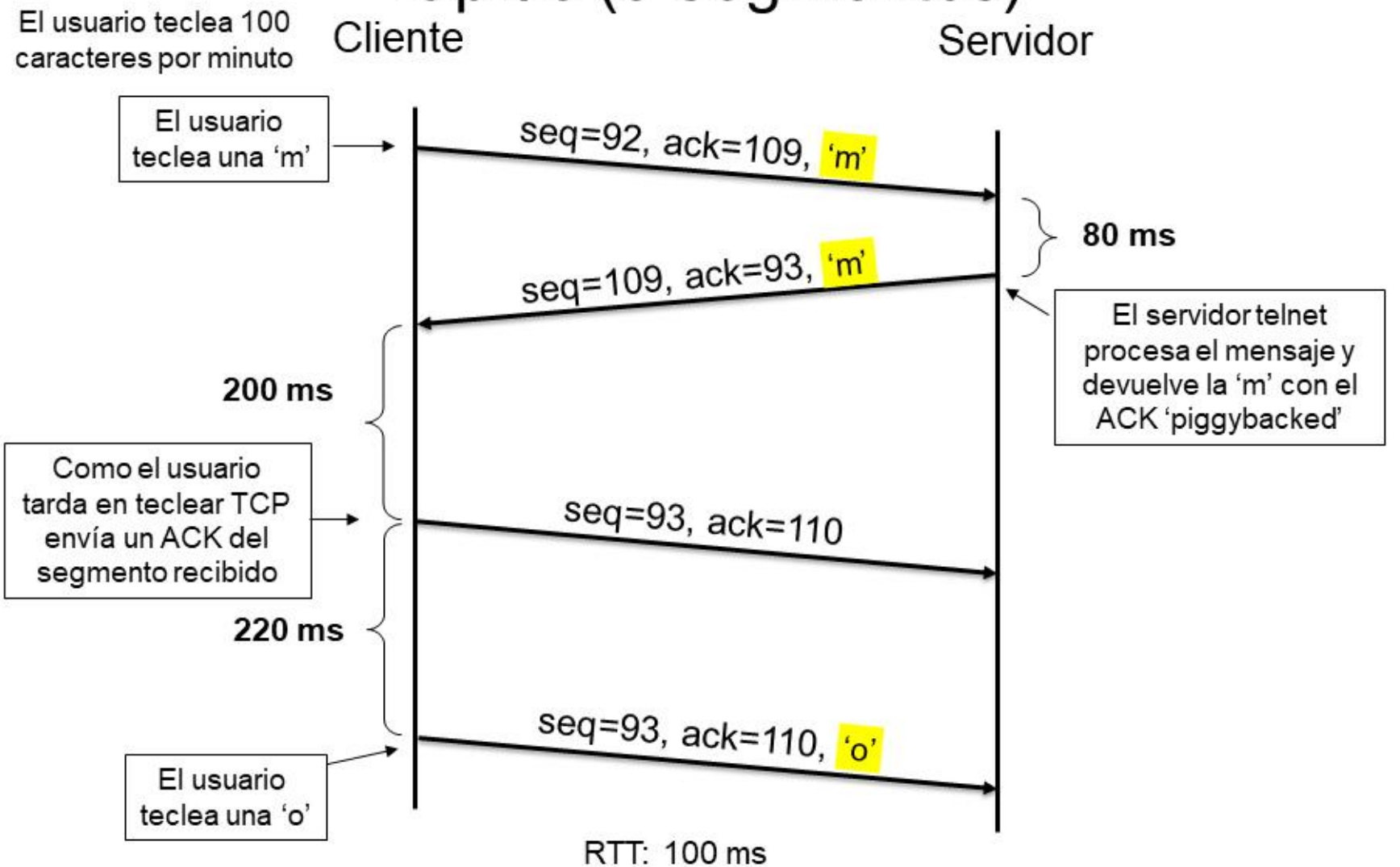
Caso 1: usuario y servidor rápidos (2 segmentos)



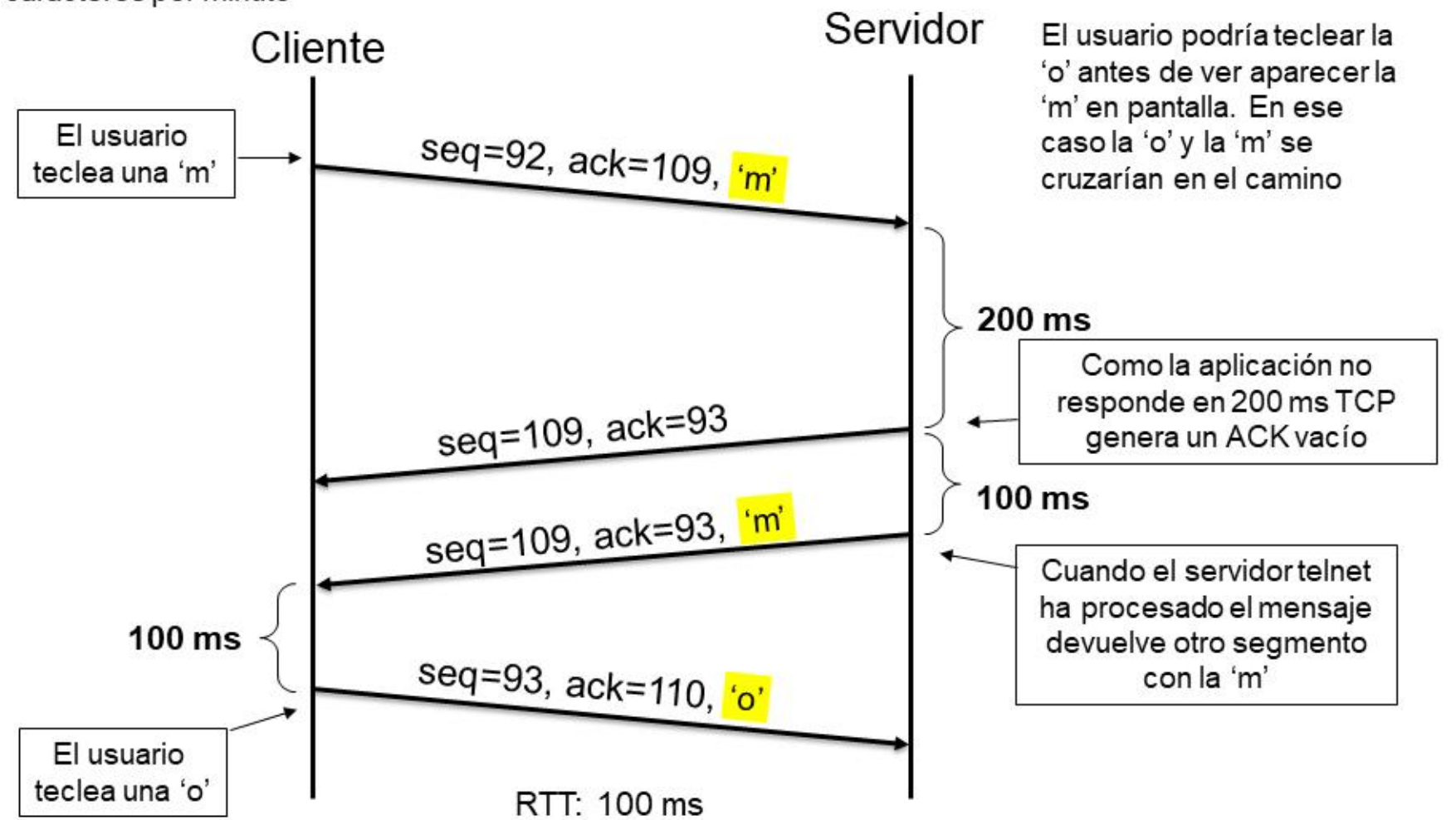
Timer de ACK retrasado

- Cuando TCP recibe un segmento con datos y no tiene nada para enviar de vuelta no devuelve el ACK enseguida, en su lugar se espera un poco a ver si la aplicación genera datos y así aprovecha para enviar juntos el ACK y los datos. Decimos que en ese caso el ACK va 'piggybacked'.
- Ese tiempo de espera se conoce como 'timer de ACK retrasado'. En muchos sistemas suele ser de unos 200 ms.
- Si se agota el timer sin que la aplicación haya producido datos se envía el ACK en un segmento sin datos
- Si el usuario o el servidor no le entregan rápido a TCP el carácter a transmitir se ha de enviar un ACK vacío

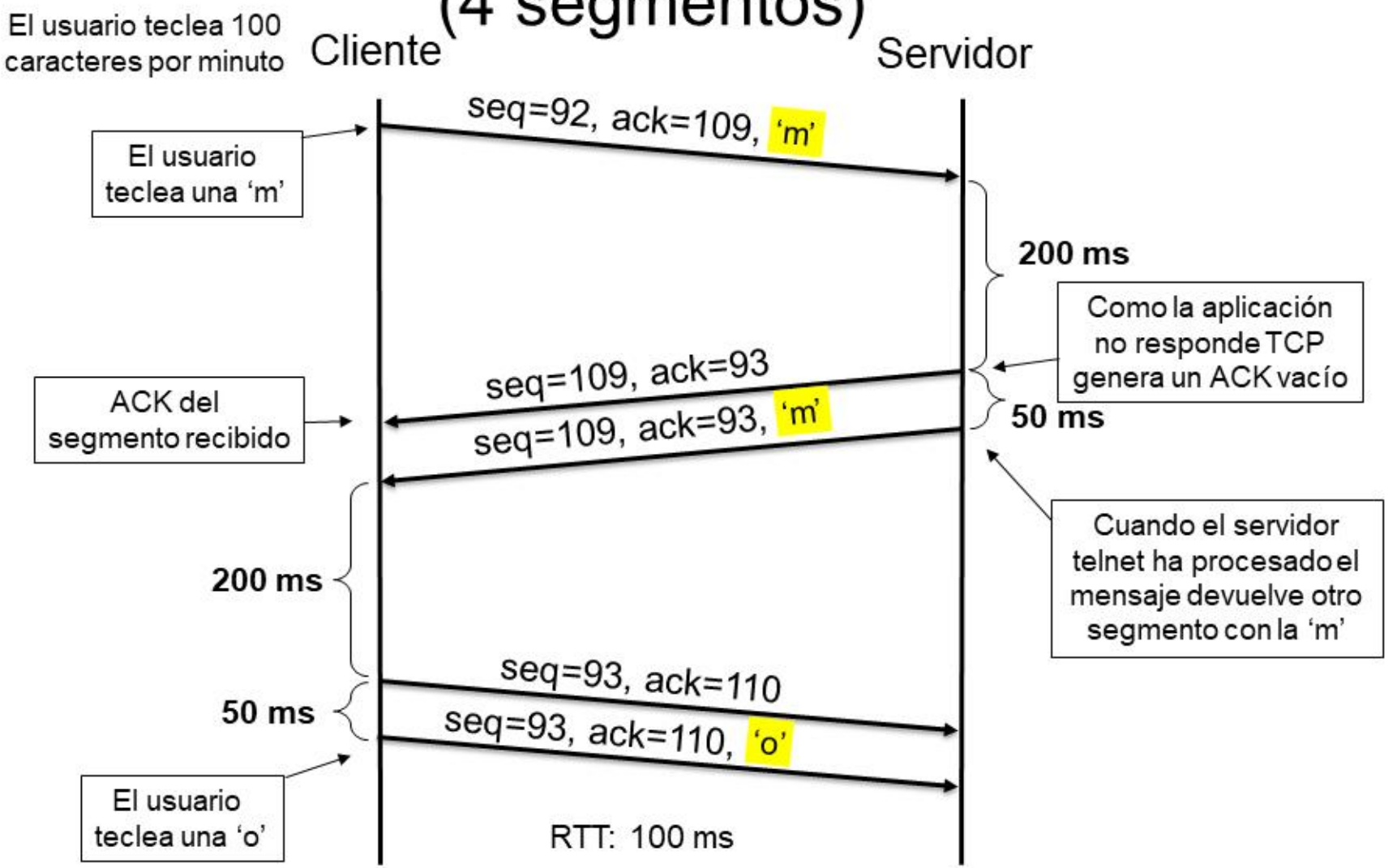
Caso 2: usuario 'patoso', servidor rápido (3 segmentos)



Caso 3: usuario rápido, servidor lento (3 segmentos)

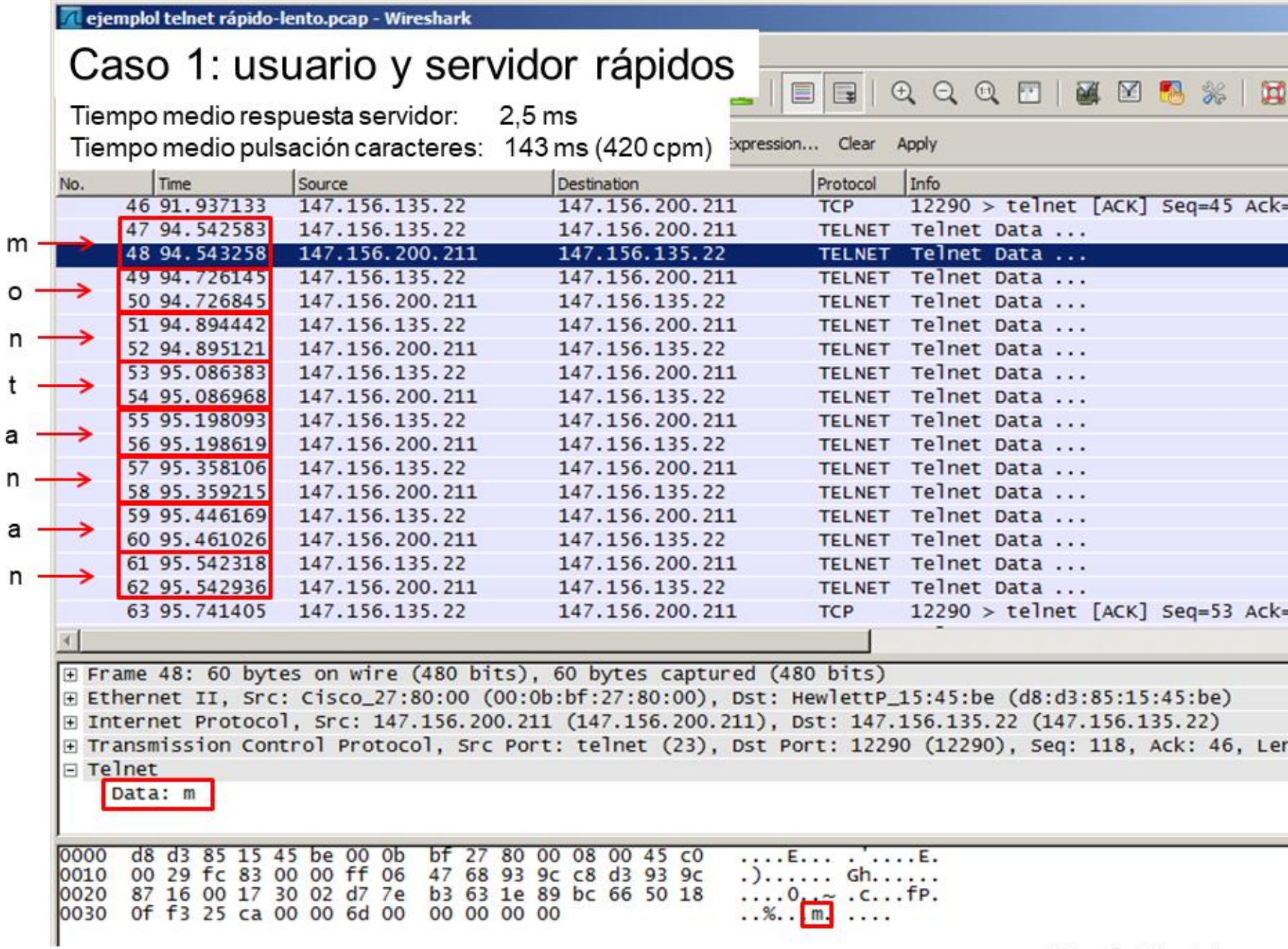


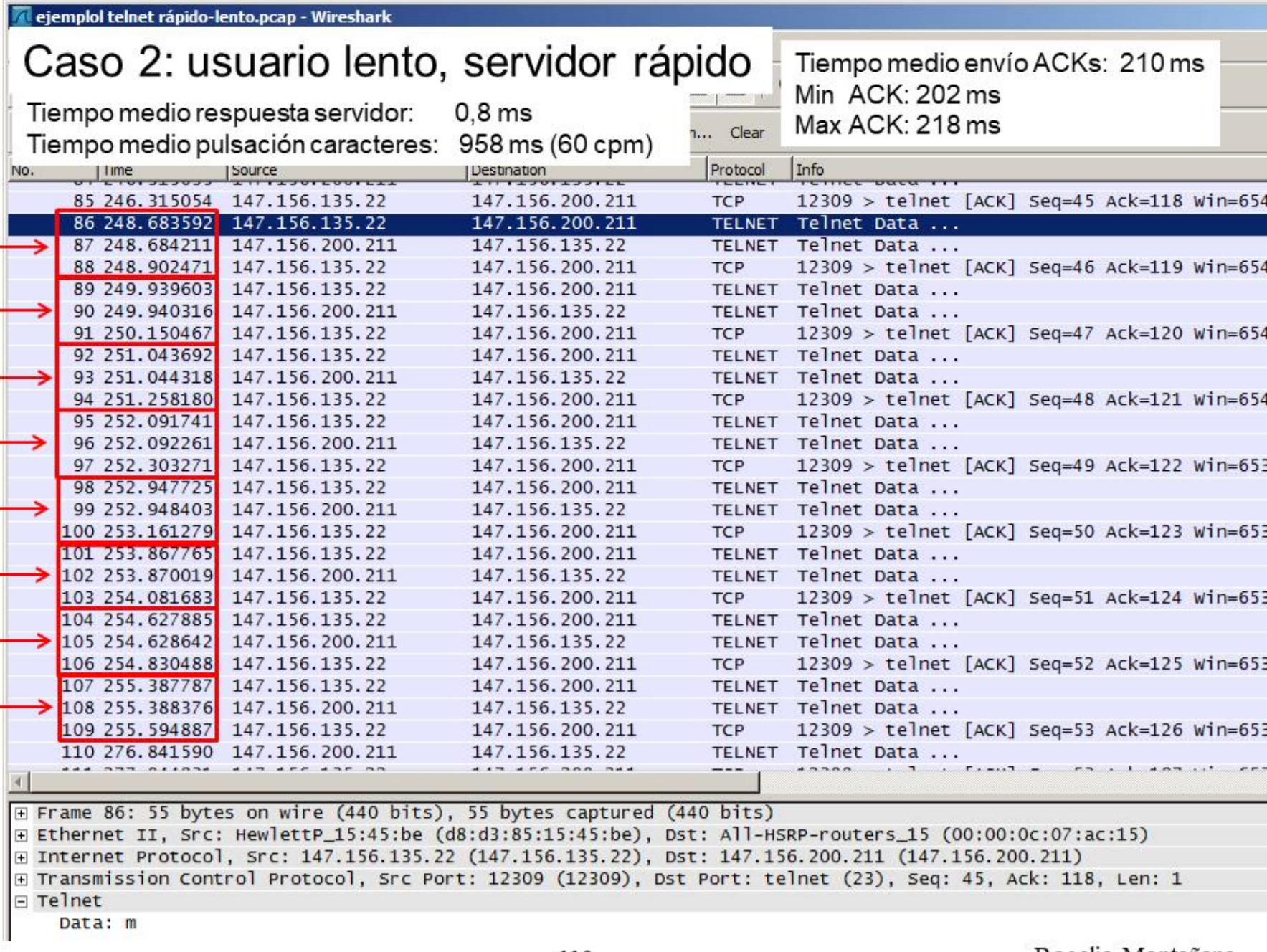
Caso 4: usuario y servidor lentos (4 segmentos)



Caso 1: usuario y servidor rápidos

Tiempo medio respuesta servidor: 2,5 ms
Tiempo medio pulsación caracteres: 143 ms (420 cpm)




ejemplo1 telnet rápido-lento.pcap - Wireshark
Caso 2: usuario lento, servidor rápido
Tiempo medio respuesta servidor: 0,8 ms
Tiempo medio pulsación caracteres: 958 ms (60 cpm)
Tiempo medio envío ACKs: 210 ms
Min ACK: 202 ms
Max ACK: 218 ms
Clear
No. Time Source Destination Protocol Info
85 246.315054 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=45 Ack=118 Win=654
86 248.683592 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
m
87 248.684211 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
88 248.902471 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=46 Ack=119 Win=654
89 249.939603 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
o
90 249.940316 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
91 250.150467 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=47 Ack=120 Win=654
92 251.043692 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
n
93 251.044318 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
94 251.258180 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=48 Ack=121 Win=654
95 252.091741 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
t
96 252.092261 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
97 252.303271 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=49 Ack=122 Win=653
98 252.947725 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
a
99 252.948403 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
100 253.161279 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=50 Ack=123 Win=653
101 253.867765 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
n
102 253.870019 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
103 254.081683 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=51 Ack=124 Win=653
104 254.627885 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
a
105 254.628642 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
106 254.830488 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=52 Ack=125 Win=653
107 255.387787 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
n
108 255.388376 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
109 255.594887 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=53 Ack=126 Win=653
110 276.841590 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
Frame 86: 55 bytes on wire (440 bits), 55 bytes captured (440 bits)
Ethernet II, Src: HewlettP_15:45:be (d8:d3:85:15:45:be), Dst: All-HSRP-routers_15 (00:00:0c:07:ac:15)
Internet Protocol, Src: 147.156.135.22 (147.156.135.22), Dst: 147.156.200.211 (147.156.200.211)
Transmission Control Protocol, Src Port: 12309 (12309), Dst Port: telnet (23), Seq: 45, Ack: 118, Len: 1
Telnet
Data: m

Cálculo del SRTT

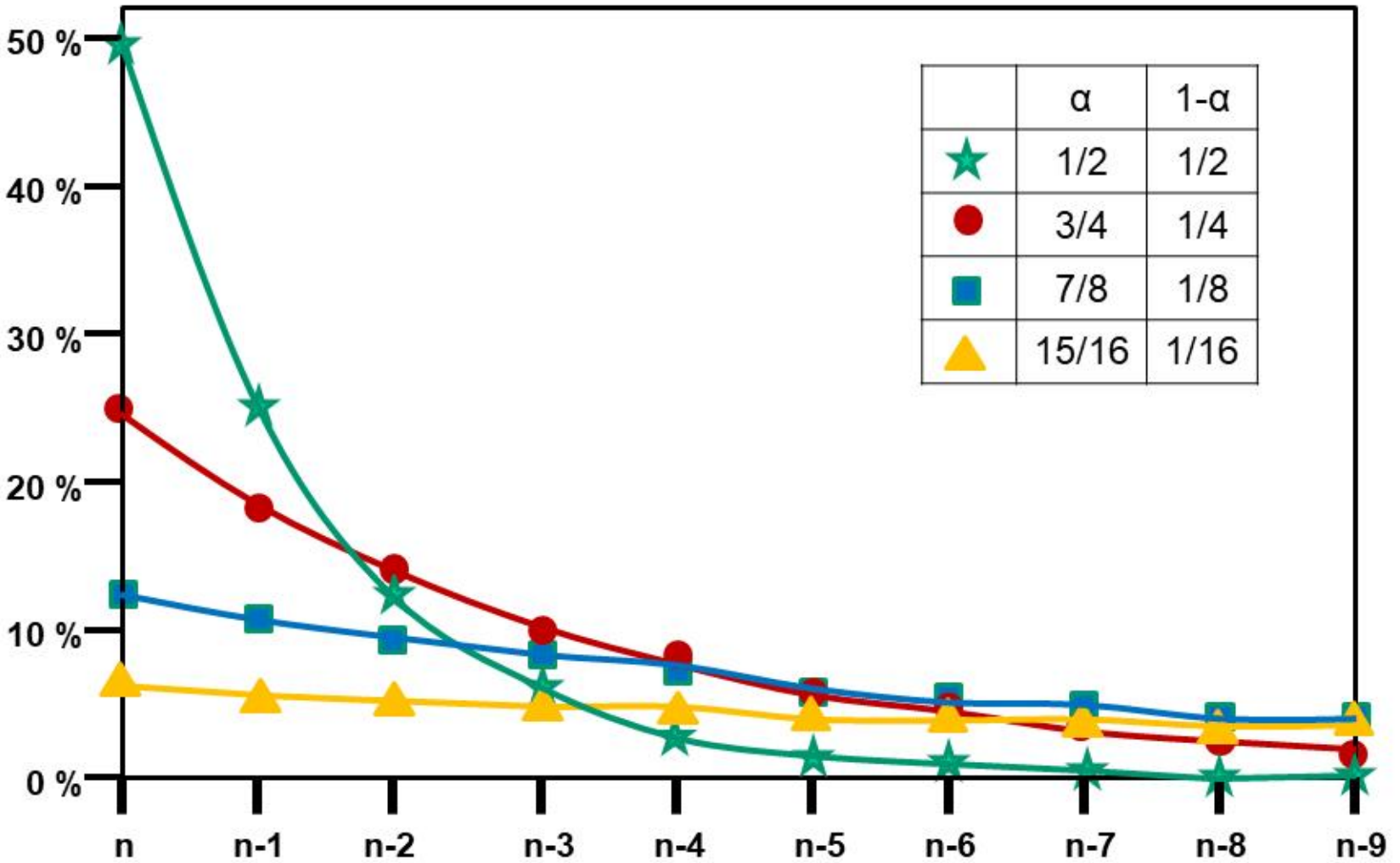
- Para el cálculo del SRTT se utiliza la fórmula:

$$SRTT_n = \alpha\ SRTT_{n-1} + (1 - \alpha)\ RTT$$

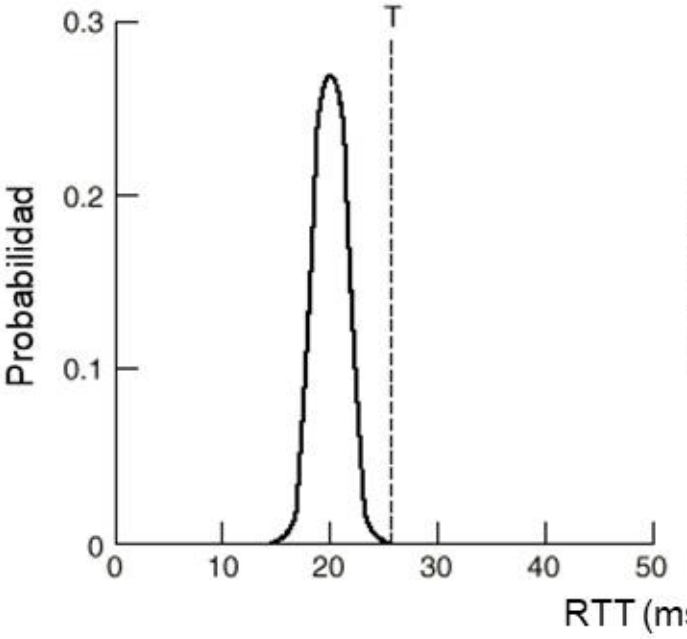
donde RTT es el último valor medido de RTT.

- α es un factor de amortiguación entre 0 y 1. Cuanto mayor es α más peso tienen los valores antiguos. Un valor de α demasiado pequeño nos impedirá responder con rapidez a situaciones cambiantes, un valor excesivo nos hará demasiado reactivos, pudiendo dar lugar a situaciones oscilantes.
- Por eficiencia del cálculo se busca que $1-\alpha = 1/(2)^n$. De ese modo la multiplicación se reduce a un desplazamiento de bits.

Peso de RTT_n en el cálculo de SRTT según α

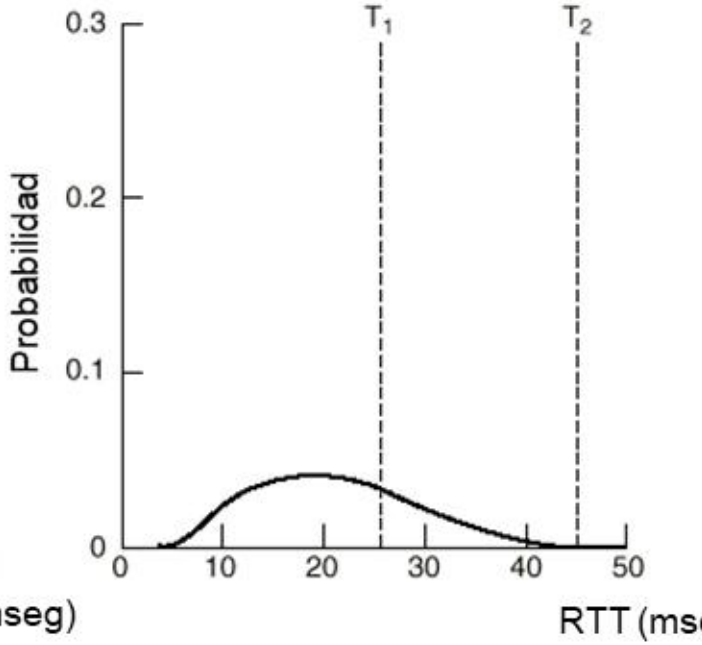


Dispersión del RTT



RTT en una red sin tráfico

Con poca dispersión el timeout puede ser solo ligeramente mayor que el valor medio



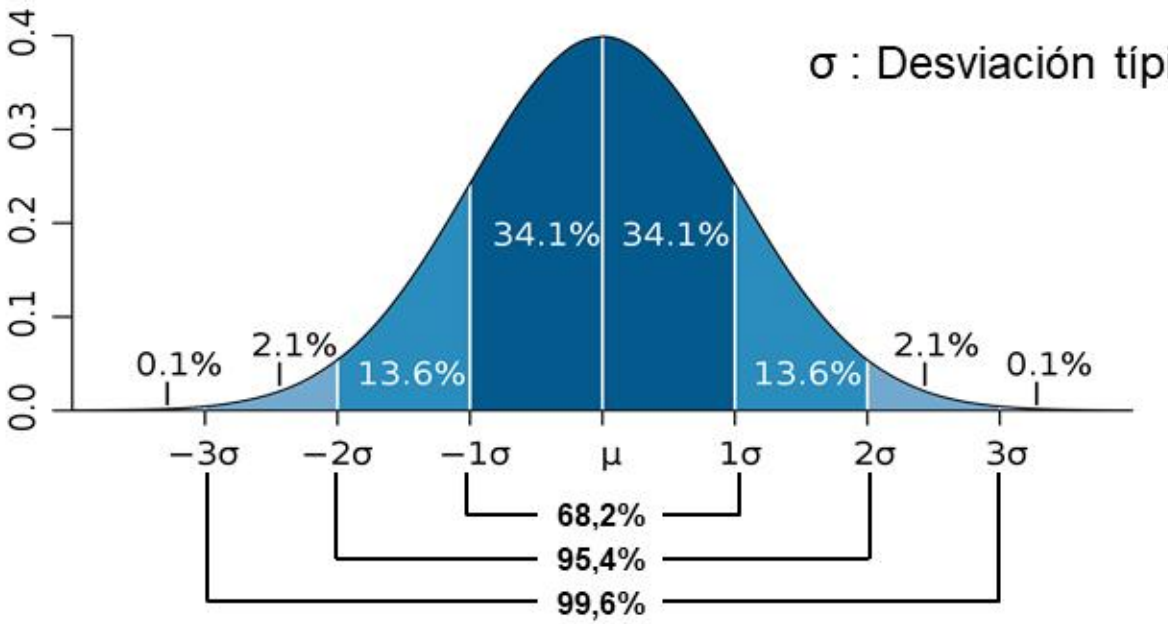
RTT en una red congestionada

Con mucha dispersión el timeout ha de ser bastante superior al valor medio

Saber el rango de variación de los RTT es tan importante como saber su valor medio. Esto se estima con la

Distribución normal

- Si los valores de RTT siguieran una distribución normal el rango MRTT ± σ abarcaría el 68% de los valores medidos, MRTT ± 2σ comprendería el 95% y MRTT ± 3σ el 99,6%:



Esto es sólo una aproximación, ya que los valores de RTT no siguen una distribución normal (la distribución no es simétrica, por ejemplo)

Variación del RTT

- A partir de los valores de RTT podemos calcular la **desviación típica (σ)** de una conexión; esto nos da una medida de la dispersión de los valores:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n} (RTT_i - MRTT)^2}{n}}$$

- En TCP se utiliza la **desviación media (D_m)**, más fácil de calcular que σ :

$$D_m = \frac{\sum_{i=1}^{n} | RTT_i - MRTT |}{n}$$

Cálculo de la variación de RTT

- Tan importante como tener una buena estimación del RTT es tener una buena estimación de su desviación media (el rango de variación de fluctuación)
- En TCP la desviación media D_m se estima a partir de la RTTVAR (RTT Variation) que se calcula como una media móvil exponencial ponderada de las diferencias (en valor absoluto) entre RTT y el SRTT (no el MRTT):

$$RTTVAR_n = \beta\, RTTVAR_{n-1} + (1 - \beta)\,|\,SRTT_{n-1} - RTT|$$

En este caso el factor de amortiguación β suele valer 3/4

Cálculo del timeout

- A partir de $SRTT_n$ y $RTTVAR_n$ se calcula el RTO (Retransmission TimeOut) como:

$$RTO = SRTT_n + 4 * RTTVAR_n$$

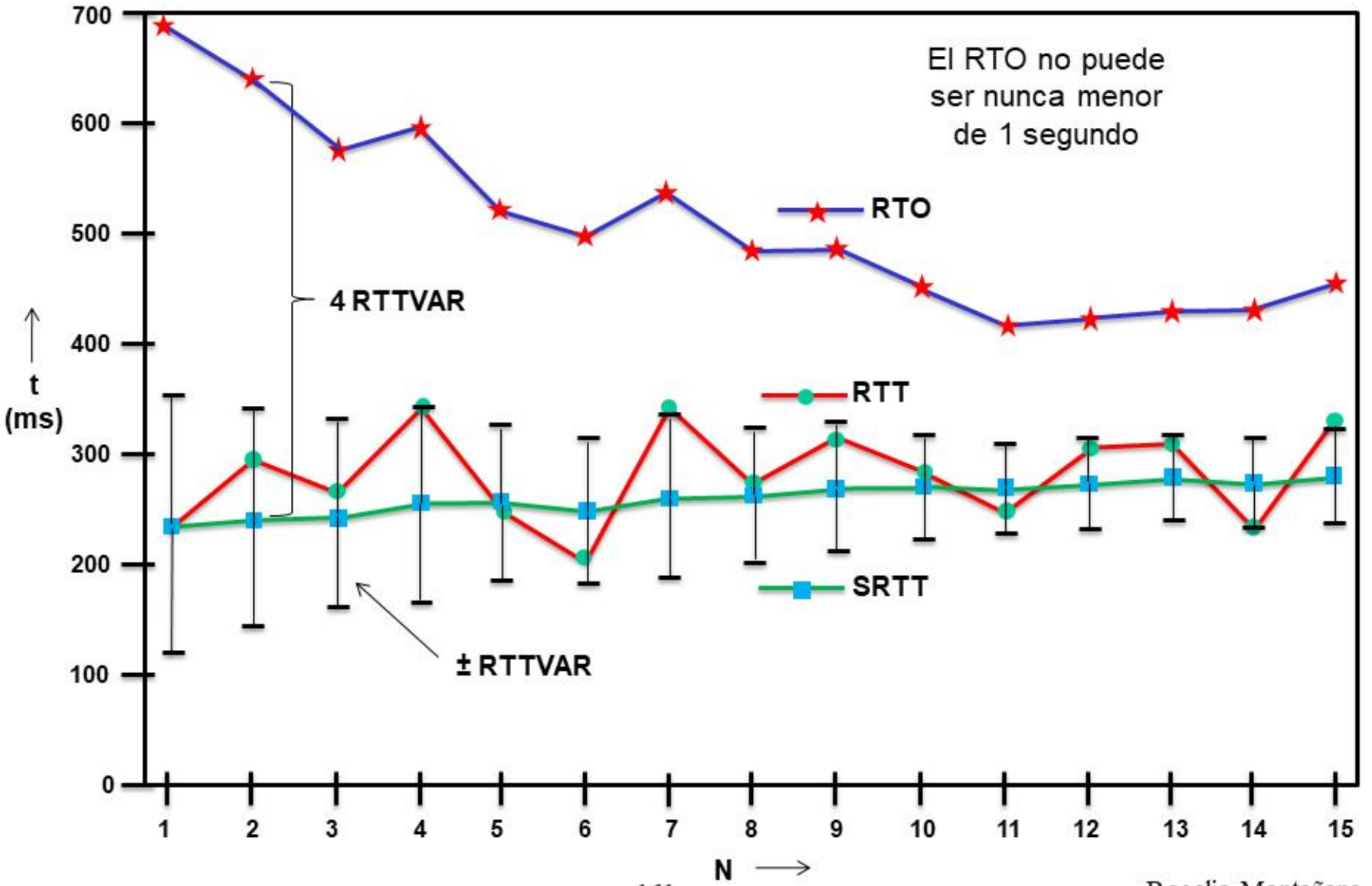
- Con este valor la media de fallos (paquetes dados por perdidos que luego aparecen) es menor del 1%.
- Todos los cálculos que tienen que ver con TCP se basan en potencias enteras de dos:
  - 1/8, en SRTT
  - 1/4 en RTTVAR
  - 4 en RTO

a fin de acelerar el cálculo (las multiplicaciones o divisiones por potencias de 2 se hacen por desplazamiento de bits).

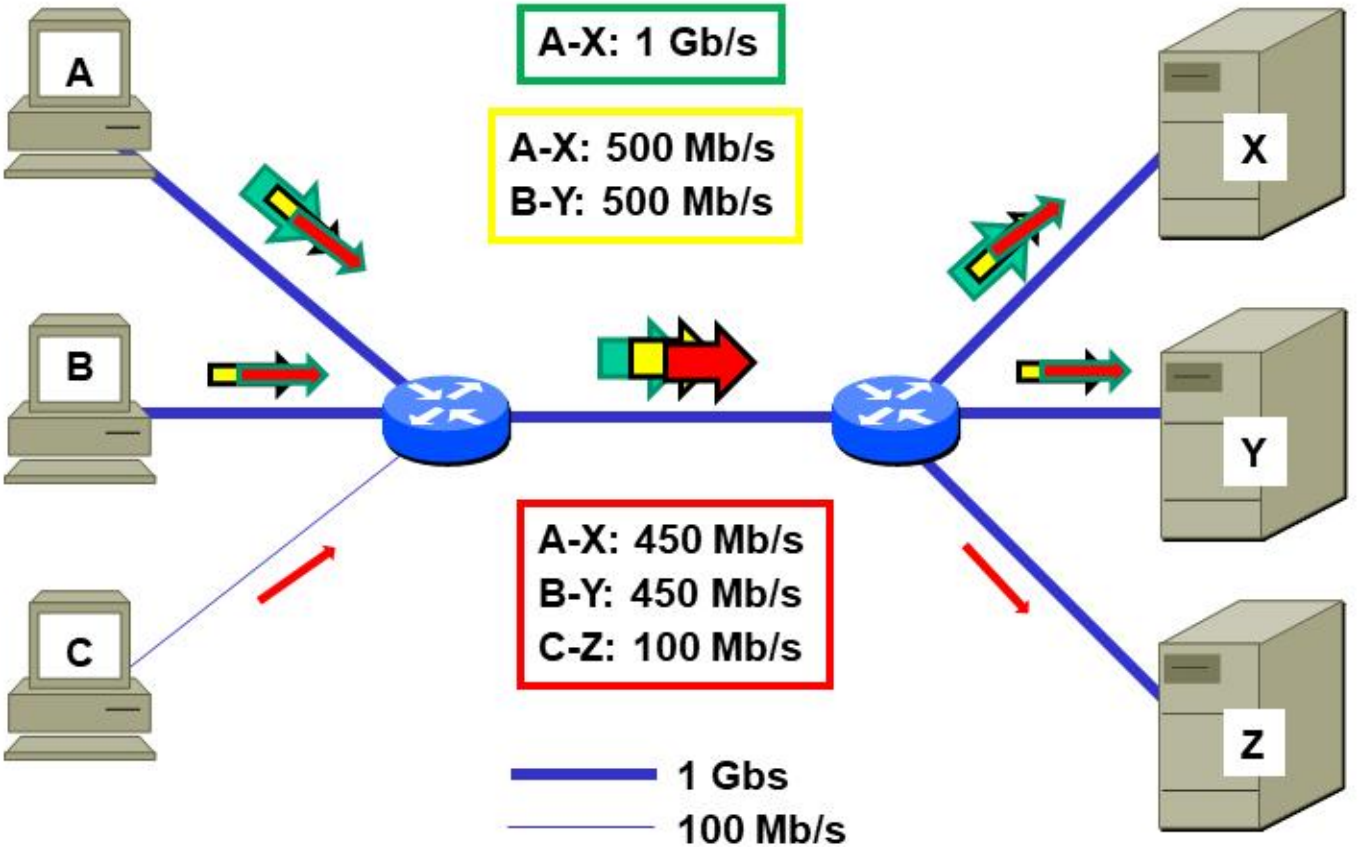
Evolución de SRTT, RTTVAR y RTO según RTT

N	RTT_n	$SRTT_n$	$RTTVAR_n$	RTO
1	230	230,00	115	690,00
2	294	238,00	102,25	647,00
3	264	241,25	83,19	574,00
4	340	253,59	87,08	601,91
5	246	252,64	67,21	521,47
6	201	246,19	63,32	499,45
7	340	257,92	70,94	541,68
8	272	259,68	56,73	486,58
9	311	266,09	55,38	487,59
10	282	268,08	45,51	450,12
11	246	265,32	39,65	423,93
12	304	270,16	39,41	427,79
13	308	274,89	39,02	430,96
14	230	269,27	40,48	431,21
15	328	276,62	45,04	456,79

Evolución de RTT, SRTT, RTTVAR y RTO



Ejemplo de reparto equitativo



Ejercicio: control de congestión

- Una conexión TCP sobre un medio físico poco fiable pierde exactamente una trama de cada 10. El nivel de enlace descarta las tramas erróneas, sin pedir reenvío (CRC):
  - Detalle como se producirá el envío de los segmentos y como evolucionará la ventana de congestión en el TCP emisor (se supone que se ha negociado utilizar SACK)
  - Estime cualitativaente la merma de rendimiento que cabe esperar debido a la tasa de error:
    A. Menor del 10%
    B. Alrededor del 10%
    C. Mayor del 10%
  - Diga como influiría el RTT en el rendimiento

Ejercicio: Evolución ventana de congestión

	cwnd (KB)	Trama	Segmento	Umbral peligro (KB)
S.S.	1	1	1	64
	2	2,3	2,3	64
	4	4,5,6,7	4,5,6,7	64
	8	8,9,10,11,12,13,14,15	8,9,10,11,12,13,14,15	64
S.S.	1	16	10	4
	2	17,18	16,17	4
	4	19,20,21,22	18,19,20,21	4
S.S.	1	23	19	2
	2	24,25	22,23	2
C.A.	3	26,27,28	24,25,26	2
	4	29,30,31,32	27,28,29,30	2
S.S.	1	33	28	2
	2	34,35	31,32	2
C.A.	3	36,37,38	33,34,35	2
	4	39,40,41,42	36,37,38,39	2
S.S.	1	43	37	2
	2	44,45	40,41	2
C.A.	3	46,47,48	42,43,44	2
	4	49,50,51,52	45,46,47,48	2
S.S.	1	53	46	2
	2	54,55	49,50	2

Ejercicio: solución

- La merma de rendimiento será siempre superior al 10%, ya que además de tener que reenviar el 10% de los paquetes tenemos que estar continuamente reiniciando la ventana de congestión.
- Cuanto mayor sea el RTT mayor será la pérdida. Con un RTT cero la pérdida sería del 10%

Problema de TCP con las redes LFN

- La ventana de control de flujo en la cabecera TCP tiene 16 bits. Su valor máximo es 65.535.
- Por tanto en TCP no es posible enviar más de 65.535 bytes seguidos sin recibir un ACK
- Si en una red BW*RTT > 64 Kbytes el rendimiento está limitado por el tamaño de ventana. La limitación es tanto mayor cuanto mayor es el BW*RTT de la red
- La limitación se aplica a cada conexión TCP por separado, si hay varias en paralelo pueden usar más capacidad

Solución al problema de TCP con las redes LFN

- Puesto que el campo tamaño de ventana no se puede ampliar los dos TCPs acuerdan no contar los bytes de uno en uno, sino de dos en dos, de cuatro en cuatro, etc. Siempre en potencias enteras de dos (es como añadir ceros por la derecha).
- Esto es lo que se conoce como factor de escala y para poder usarlo se ha de negociar en el momento de conectarse. Se negocia en una opción del segmento SYN, ha de estar soportado por los dos TCP y si se acuerda se ha de mantener durante toda la vida de la conexión

Ejemplo de uso del factor de escala

- El IFIC (Instituto de Física Corpuscular) realiza transferencias de datos masivas entre Valencia y el CERN en Ginebra, Suiza
- En pruebas hechas con un FTP entre máquinas conectadas a 100 Mb/s se obtenía un caudal máximo de 9 Mb/s
- Se observó que lanzando varias transferencias en paralelo se obtenían caudales superiores, pudiendo acercase a los 100 Mb/s
- Esto hizo sospechar que la limitación no se debía al ancho de banda de la conexión, sino al BW*RTT

```
C:\Documents and Settings\montanan>ping www.cern.ch

Haciendo ping a webr2.cern.ch [137.138.28.230] con 32 bytes de datos:

Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114

Estadísticas de ping para 137.138.28.230:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0 (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 43ms, Máximo = 43ms, Media = 43ms


C:\Documents and Settings\montanan >tracert www.cern.ch

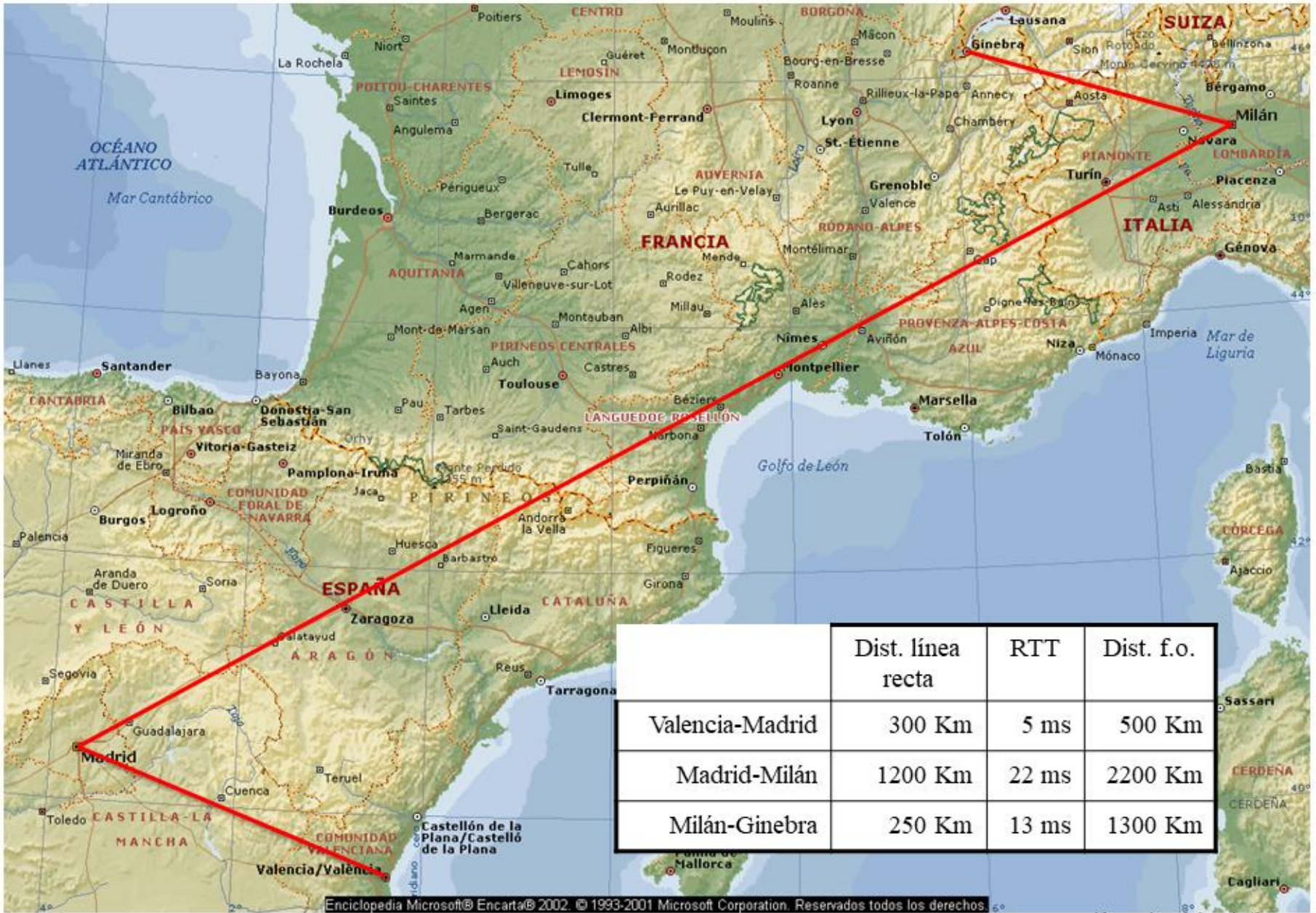
Traza a la dirección webr2.cern.ch [137.138.28.230] sobre un máximo de 30 saltos:

  1    <1 ms    <1 ms    <1 ms  burl3ci.red.uv.es [147.156.2.50]
  2    <1 ms    <1 ms    <1 ms  burladerol3.red.uv.es [147.156.200.184]
  3    <1 ms    <1 ms    <1 ms  neveraout.red.uv.es [147.156.7.1]
  4    <1 ms    <1 ms    <1 ms  AT0-2-0-0.EB-Valencia0.red.rediris.es [130.206.211.181]
  5     6 ms     6 ms     6 ms  VAL.SO3-0-0.EB-IRIS4.red.rediris.es [130.206.240.13]
  6     7 ms     7 ms     7 ms  rediris.es1.es.geant.net [62.40.103.61]
  7    29 ms    29 ms    29 ms  es.it1.it.geant.net [62.40.96.186]
  8    42 ms    42 ms    42 ms  it.ch1.ch.geant.net [62.40.96.33]
  9    43 ms    42 ms    42 ms  swiCE2-P6-1.switch.ch [62.40.103.18]
 10    43 ms    43 ms    42 ms  cernh7-gb1-1.cern.ch [192.65.184.222]
 11    42 ms    43 ms    42 ms  cernh2-vlan2.cern.ch [192.65.192.2]
C:\Documents and Settings\montanan>
```

5 ms { 4 5

22 ms { 6 7

13 ms { 8



	Dist. línea recta	RTT	Dist. f.o.
Valencia-Madrid	300 Km	5 ms	500 Km
Madrid-Milán	1200 Km	22 ms	2200 Km
Milán-Ginebra	250 Km	13 ms	1300 Km

Rendimiento de la conexión TCP

- Caudal máximo = ventana / RTT
- Con RTT = 43 ms y ventana 524.288 bits:
  Caudal máximo = 524.288 / 0,043 = **12,2 Mb/s**
- Para mejorar el rendimiento se configuró el uso de factor de escala en TCP, obteniendo así un mayor tamaño de ventana. Con un factor 16x ya se conseguían casi los 100 Mb/s (velocidad de la interfaz):

Factor de escala	Tam. Ventana (bits)	Caudal max. (Mb/s)
0 (1x)	524.288	12,2
1 (2x)	1.048.576	24,4
2 (4x)	2.097.152	48,8
3 (8x)	4.194.304	97,5
4 (16x)	8.388.608	195,1

Factor de escala

- El factor de escala puede añadir como máximo 14 ceros, equivalente a una ventana de $2^{30} = 1$ GB.
- Con un RTT de 86 ms y un caudal de 100 Gb/s ese tamaño de ventana ya resulta insuficiente
- Hay que destacar no obstante que la limitación se aplica a cada conexión TCP individual e independientemente
- El factor de escala se ha de fijar a ciegas en la conexión, no existe un mecanismo de 'descubrimiento del factor de escala del trayecto'

Ejercicio: redes LFN

P: Sesión TCP con 100 Mb/s de BW y 20 ms de RTT. Calcular caudal máximo aprovechable suponiendo que el factor de escala no está disponible.

R: De la fórmula: Ventana = BW * RTT

obtenemos: BW = Ventana / RTT

Sustituyendo: BW = 65.535*8 / 0,02 = 26,2 Mb/s